DSL Grammar Documentation for Simulation Sequencing

Sreejith P. Sajeev

April 14, 2026

1 Introduction

This document details the Domain-Specific Language (DSL) for sequencing events in a power system simulator, based on the `PowerEventDSL.g4` grammar. **Implemented using ANTLR**, this DSL provides robust parsing for defining simulation parameters and scheduling events. Key features include:

- **Simulation Parameters**: Set duration and time steps.
- **Time-Based Events**: Trigger actions at specific times, at `now`, or over intervals.
- **Immediate Events**: Actions without an `at` clause execute at the next step (`now`).
- **Absolute Setpoints**: Set load/generator to `P/Q/U` with optional `from` for ramps; signed values supported.
- **Condition-Based Events**: Respond to dynamic system states and measurement channels (`voltage`, `frequency`, `P`, `Q`, `U`, `state`).
- **Dependent Events**: Link actions to prior tagged events.
- **Targeting**: Single, list, static `cluster`, and `dynamic cluster` targets; dynamic filters (`regex`, `group by`, `where`, device on qualifiers).
- **Line Actions**: Open/close transmission lines directly by IIDM identifier.
- **Network Actions**: Fine-grained `change network characteristics` (link/node) with `drop`, `delay`, and `corruption` operators.
- **Print Actions**: Log informational comments into the session trace using `print`; does not modify system state.

All statements end with a semicolon (`;`) for well-defined syntax.

2 Simulation Parameters

2.1 Set Simulation Duration

Syntax:

```
set simulation duration to <NUMBER> <TIME_UNIT>;
```

Example:

```
set simulation duration to 30 seconds;
```

Description: Defines the total simulation runtime. `<TIME_UNIT>` can be `second(s)`, `minute(s)`, or `hour(s)`.

2.2 Set Time Step

Syntax:

```
set time step to <NUMBER> <TIME_UNIT>;
```

Example:

```
set time step to 1 seconds;
```

Description: Specifies the interval between simulation updates. Time units accept singular or plural forms.

3 Event Definitions

Events describe *when* and *under what conditions* actions are executed.

3.1 Time Expressions

Syntax:

```
<TIME_EXPR> ::= <NUMBER> <TIME_UNIT>
              | now
              | now + <NUMBER> <TIME_UNIT>
              | now - <NUMBER> <TIME_UNIT>

<TIME_SPEC> ::= <TIME_EXPR>
              | [<TIME_EXPR>, <TIME_EXPR>] [with law <LAW_SPEC>]
```

Description: `now` resolves to the next simulator step (`current_time` + `time_step`).

3.2 Point-in-Time Event

Syntax:

```
at <TIME_SPEC>, <ACTION> [tagged "<TAG_ID>"];
```

Example:

```
at 5 seconds, open switch "S1" tagged "event1";
at now + 10 seconds, close switch "S1";
```

3.3 Immediate Event

Syntax:

```
<ACTION> [tagged "<TAG_ID>"];
```

Example:

```
open switch "S1";
```

Description: Executes at `now` (next simulator step).

3.4 Interval-Based Event

Syntax:

```
at [<TIME_EXPR>, <TIME_EXPR>] [with law <LAW_SPEC>], <ACTION>;
```

Example:

```
at [now, now + 20 seconds] with law "linear", decrease load "B3" by
   10%;
at [7 seconds , 14 seconds ] with law " linear " , decrease load "
   B3 " by 10%;
```

Description: Applies an action across a time interval. Law can be `linear` or a quoted string (custom). Linear is the only built-in scaling law.

3.5 Condition-Based Event

Syntax:

```
when <VARIABLE> at <COMPONENT_TYPE> "<COMPONENT_ID>" <OPERATOR> <
   NUMBER> <UNIT>, <ACTION>;
```

Examples:

```
when voltage at bus "Bus1" < 0.9 pu, increase load "Load1" by 10%;
when frequency at voltageLevel "_BUS____2_VL" > 50.1 Hz, decrease
   generator "G1" by 5%;
when P at line "L1" > 8 pu, decrease generator "G2" by 5%;
when state at asset "BreakerA" == 0, close switch "CB5";
```

Description: Evaluated every step; triggers once when the predicate becomes true (unless configured as persistent). Supported `<COMPONENT_TYPE>` include `bus`, `line`, `asset`, and `voltageLevel` (IIDM context).

3.6 Dependent Event

Syntax:

```
after "<TAG_ID>" is completed, <ACTION>;
```

Example:

```
after "event1" is completed, apply fault at line "L1";
```

Description: Executes the action on the first step after the referenced tag completes.

3.7 Tags and Dependency Semantics

What a tag represents:

- A tag is a symbolic label attached with `tagged "<TAG_ID>"` to name one event declaration.
- Tags are synchronization and traceability handles; they do not change the control action payload itself.

How tags are implemented:

- Each parsed event receives an internal unique event id (UUID).
- When an event is tagged, the interpreter stores a label mapping `<TAG_ID> -> event id`.
- `after "<TAG_ID>" is completed` resolves the label to an internal dependency id; unknown tags raise a semantic error.
- Dependency events are promoted once the upstream event status becomes completed. Completion follows event type semantics: point/immediate events complete on execution, interval events complete at interval end, and condition events complete on first trigger.

How tags are intended to be used:

- Tag milestone events that other events must wait for.
- Use unique, descriptive names (for example `"line_disconnect"` or `"voltage_control"`) to keep dependency chains readable.
- Keep one meaning per tag and avoid reusing the same label across unrelated phases.
- For multi-phase workflows, define explicit tags for each phase boundary instead of relying on implicit timing.

4 Action Types

4.1 Switch Actions

Syntax:

```
open switch "<SWITCH_ID>";
close switch "<SWITCH_ID>";
```

Effect: Disconnect or connect network elements via breakers/switches.

4.2 Line Actions

Syntax:

```
open line "<LINE_ID>";
close line "<LINE_ID>";
```

Examples:

```
at 5 seconds, open line "LINE_B_A_1" tagged "line_disconnect";
at 15 seconds, close line "LINE_B_A_1";
when voltage at voltageLevel "VL_A_225" < 0.95 pu, open line "
   LINE_C_A_1";
after "line_disconnect" is completed, decrease load "LOAD1" by 5%;
```

Description: Disconnect or connect transmission lines directly by their IIDM identifier. Uses `state` attribute with values `0` (open/disconnected) and `1` (close/connected). These values can be customized via the `ControlValueResolver` configuration or environment variable `CONTROL_VALUE_MAP`.

4.3 Load Actions

Syntax:

```
increase load <TARGET> by <NUMBER>%;
decrease load <TARGET> by <NUMBER>%;
```

Targets: single ID, list, cluster, dynamic cluster with filters.

4.4 Generator Actions

Syntax:

```
increase generator <TARGET> by <NUMBER>%;
decrease generator <TARGET> by <NUMBER>%;
```

Effect: Adjust generation setpoints; may interact with control loops.

4.5 Set Actions (Absolute Values and Ramps)

Syntax:

```
set load <TARGET> [from <SIGNED_NUMBER>] to <SIGNED_NUMBER> [P|Q|U];
set generator <TARGET> [from <SIGNED_NUMBER>] to <SIGNED_NUMBER> [P|
   Q|U];
```

Examples:

```
set generator "G1" to 900 P;
set generator "G1" from 700 to 900 P;
set load "L1" to -100 Q;
```

Description:

- Point events set the absolute value immediately.
- Interval events with law `"linear"` ramp the absolute value.
- If `from` is omitted on a linear interval, the ramp starts from the current measured value.
- `<SIGNED_NUMBER>` accepts an optional `+` or `-`.

4.6 Fault Actions

Syntax:

```
apply fault at <COMPONENT_TYPE> "<COMPONENT_ID>";
clear fault at <COMPONENT_TYPE> "<COMPONENT_ID>";
```

Effect: Insert/remove short-circuit conditions or equivalent models.

4.7 Print Action

Syntax:

```
print "<MESSAGE>";
```

Examples:

```
at 5 seconds, print "Raising load in Area A";
at now, print "Starting fault injection sequence";
when voltage at voltageLevel "_BUS____2_VL" < 0.95 pu, print "
   Undervoltage detected at Bus 2";
after "event1" is completed, print "Phase 1 complete, beginning
   recovery";
print "Immediate annotation at next step";
```

Description:

- Logs an informational text comment into the session trace.

- Does not modify the system state - it is purely an annotation mechanism.
- The message is recorded in the session event store and included in the action output with `actionType: "print"`.
- During session replay, `print` entries appear in chronological order alongside other actions, helping explain the intent or context of subsequent operations.
- Can be used with all event triggers: time-based (`at`), immediate, condition-based (`when`), and dependent (`after`).

4.8 Network Actions

These actions modify underlying network/telemetry characteristics (IIDM-compatible), at either link (edge) or node (vertex) scope.

General Syntax

```
change network characteristics
   [link from "<SRC_ID>" to "<DST_ID>"]?
   [node "<NODE_ID>"]*
   (<OPERATOR_CLAUSE>)+ ;
```

Multiple operator clauses can be composed in a single statement.

Operators

- `drop`: packet/measurement loss

```
drop fixed <true|false>
drop random <PROBABILITY>
```

- `delay`: latency injection

```
delay fixed <NUMBER> seconds
delay gauss mean <NUMBER> seconds stddev <NUMBER> seconds
```

- `corruption`: data perturbation

```
corruption none [likelihood <PROBABILITY>]
corruption random_noise_measurement [likelihood <PROBABILITY>]
   range <LOW> to <HIGH>
corruption random_noise_control [likelihood <PROBABILITY>]
   range <LOW> to <HIGH>
corruption shift_measurement [likelihood <PROBABILITY>] by <
   NUMBER>
corruption zero_out_measurement [likelihood <PROBABILITY>] [
   value <NUMBER>]
corruption invert_control [likelihood <PROBABILITY>] [by <
   NUMBER>]
```

Examples

```
at 1 seconds, change network characteristics
  link from "CC" to "substation_1",
  drop fixed true
  tagged "CC_to_substation_1";

after "CC_to_substation_1" is completed, change network
   characteristics
  link from "CC" to "substation_2",
  link from "CC" to "substation_3",
  node "substation_2",
  delay fixed 0.15 seconds;

at 6 seconds, change network characteristics
  link from "CC" to "substation_5", node "substation_5", node "
   substation_4",
  corruption shift_measurement likelihood 0.2 by 0.02
  tagged "substation_5_shift";

at 8 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  delay gauss mean 0.15 seconds stddev 0.1 seconds;

at 8 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  corruption none likelihood 0.2;

at 8 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  corruption zero_out_measurement;

at 10 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  corruption invert_control;
```

5 Target Specifications

5.1 Single Target

Syntax: `"<ID>"` **Example:** `"B3"`

5.2 List Target

Syntax: `[<ITEM>, <ITEM>, ...]` **Example:** `["B1", "B2"]`

5.3 Cluster Target

Syntax: `cluster ["<ID>", "<ID>", ...]`

5.4 Dynamic Cluster Target with Filters

Purpose: Resolve device groups at runtime using metadata/topology.

```
dynamic cluster regex "<REGEX>" on <DEVICE_KIND>
dynamic cluster group by "<ATTRIBUTE>" "<VALUE>" on <DEVICE_KIND>
dynamic cluster where "<PROPERTY>" <OP> <VALUE> on <DEVICE_KIND>
```

Examples:

```
at 8 seconds, increase load dynamic cluster regex "LOAD.*" on load
   by 10%;

at 22 seconds, decrease load dynamic cluster
   group by "voltage_level_id" "_BUS____2_VL" on load by 15%;

at 28 seconds, increase generator dynamic cluster
   where "target_p" > 50 MW on generator by 10%;

at 30 seconds, increase load dynamic cluster
   group by "substation_id" "_BUS____4_SS" on load by 12%;
```

6 Error and Corruption Types

6.1 Syntax Errors

Missing semicolons, invalid tokens, or misordered keywords. Detected by ANTLR.

6.2 Semantic Errors

Invalid identifiers, unit mismatches (e.g., comparing voltage to Hz), or intervals where start $\geq$ end.

6.3 Runtime Errors

Unresolved dynamic clusters, missing dependency tags, or acting on disconnected components. Network/IIDM operators validate probabilities in $[0, 1]$ and positive times.

7 ANTLR Implementation

The DSL is powered by ANTLR:

- **Grammar File**: `PowerEventDSL.g4`

- **Generated Classes**: Lexer (`PowerEventDSLLexer`), Parser (`PowerEventDSLParser`), Visitor (`PowerEventDSLVisitor`).
- **Process**: Parses DSL into an Abstract Syntax Tree (AST) for interpretation.

8 Example DSL Scripts

8.1 IEEE 14-Bus Demonstration (Time/Condition/Dependent/-Dynamic)

```
// Compatible with IEEE 14-bus network
// Demonstrates time-based, condition-based, dependent events, and dynamic clusters
// Adjusted to avoid transformer and line parsing errors

// Set simulation parameters
set simulation duration to 60 seconds;
set time step to 1 seconds;

// Time-based events
at 5 seconds, open switch "S1" tagged "switch_open";
at 15 seconds, close switch "S1";

// Line actions
at 5 seconds, open line "LINE_B_A_1" tagged "line_open";
at 20 seconds, close line "LINE_B_A_1";

// Print action: annotate the session log
at 8 seconds, print "Raising load in Area A";

// Interval-based load increase with linear law
at [10 seconds, 20 seconds] with law "linear", increase load "LOAD   2" by 20%;

// Condition-based generator adjustment
when voltage at voltageLevel "_BUS____1_VL" > 1.05 pu,
   decrease generator "GEN   1" by 10% tagged "voltage_control";

// Condition-triggered print annotation
when voltage at voltageLevel "_BUS____2_VL" < 0.95 pu,
   print "Undervoltage detected at Bus 2";

// Dependent event
after "switch_open" is completed, close switch "S2";
after "line_open" is completed, decrease load "LOAD   3" by 5%;

```

```
// Fault application and clearing
at 12 seconds, apply fault at line "_BUS____1-_BUS____2-1";
at 18 seconds, clear fault at line "_BUS____1-_BUS____2-1";

// List-based load increase
at 3 seconds, increase load ["LOAD  2", "LOAD 3", "LOAD 4"] by 15%;

// Cluster-based generator decrease
at 25 seconds, decrease generator cluster "GenCluster" ["GEN  2", "
   GEN 3"] by 5%;

// Complex condition-based load adjustment (logical AND)
when (voltage at voltageLevel "_BUS____2_VL" < 0.95 pu
       and frequency at voltageLevel "_BUS____2_VL" > 50.1 Hz),
  increase load "LOAD  2" by 5%;

// Interval-based load decrease with step law
at [7 seconds, 14 seconds] with law "step", decrease load "LOAD  3"
   by 10%;

// Dependent event after voltage control
after "voltage_control" is completed, open switch "S3";

// Dynamic clusters
at 8 seconds, increase load dynamic cluster regex "LOAD.*" on load
   by 10%;
at 22 seconds, decrease load dynamic cluster group by "
   voltage_level_id" "_BUS____2_VL" on load by 15%;
at 28 seconds, increase generator dynamic cluster where "target_p" >
    50 MW on generator by 10%;
at 30 seconds, increase load dynamic cluster group by "substation_id
   " "_BUS____4_SS" on load by 12%;
```

8.2 IIDM/Network Actions Demonstration

```
// IIDM network characteristics manipulation demo
set simulation duration to 1800 seconds;
set time step to 1 seconds;

at 1 seconds, change network characteristics
  link from "CC" to "substation_1",
  drop fixed true
  tagged "CC_to_substation_1";

after "CC_to_substation_1" is completed, change network
   characteristics
```

```
  link from "CC" to "substation_2",
  link from "CC" to "substation_3",
  node "substation_2",
  delay fixed 0.15 seconds;

at 2 seconds, change network characteristics
  node "substation_3", node "CC",
  corruption random_noise_measurement range 0.0 to 0.001
  tagged "substation_3_noise";

at 5 seconds, change network characteristics
  link from "CC" to "substation_6",
  node "substation_6",
  drop random 0.2;

when voltage at voltageLevel "VL_A_400" < 0.97 pu, change network
   characteristics
  link from "CC" to "substation_4", node "substation_4",
  drop random 0.3;

at 6 seconds, change network characteristics
  link from "CC" to "substation_5", node "substation_5", node "
   substation_4",
  corruption shift_measurement likelihood 0.2 by 0.02
  tagged "substation_5_shift";

after "substation_5_shift" is completed, change network
   characteristics
  link from "CC" to "substation_7",
  node "substation_7",
  drop fixed false;

at 8 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  delay gauss mean 0.15 seconds stddev 0.1 seconds;

at 8 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  corruption none likelihood 0.2;

at 8 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  corruption zero_out_measurement;

at 10 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  corruption invert_control;
```

8.3 Now time Expression and Set actions

```
set simulation duration to 300 seconds;
set time step to 1 seconds;

at now, open switch "VL_A_400_COUPLER_BREAKER";
at now + 10 seconds, set load dynamic cluster regex "^Y.*(T|Y|TR)
   .[12].$" on load to 50 P;
at 30 seconds, set generator "GEN_F_1" to 100 Q;
at [now, now + 20 seconds] with law "linear", set generator "GEN_F_4
   " to 900 P;
```

8.4 Line Actions Demonstration

```
// Demonstrates line open/close actions
set simulation duration to 60 seconds;
set time step to 1 seconds;

// Open a line at a specific time
at 5 seconds, open line "LINE_B_A_1" tagged "line_disconnect";

// Close the line later
at 25 seconds, close line "LINE_B_A_1";

// Condition-based line action
when voltage at voltageLevel "VL_A_225" < 0.95 pu, open line "
   LINE_C_A_1";

// Dependent action after line disconnect
after "line_disconnect" is completed, decrease load "LOAD1" by 10%;

// Multiple line operations
at 10 seconds, open line "LINE_D_A_1";
at 15 seconds, open line "LINE_E_A_1";
at 30 seconds, close line "LINE_D_A_1";
at 35 seconds, close line "LINE_E_A_1";
```

8.5 Print Actions Demonstration

```
// Demonstrates print actions for session logging
set simulation duration to 60 seconds;
set time step to 1 seconds;

// Immediate print at next step
print "Scenario start: IEEE 14-bus stress test";
```

```
7
8  // Time-based print annotations
9  at 5 seconds, print "Phase 1: Disconnecting line LINE_B_A_1";
10 at 5 seconds, open line "LINE_B_A_1" tagged "line_disconnect";
11
12 at 15 seconds, print "Phase 2: Applying fault injection";
13 at 15 seconds, apply fault at line "_BUS____1-_BUS____2-1";
14
15 // Condition-triggered print
16 when voltage at voltageLevel "_BUS____2_VL" < 0.95 pu,
17    print "WARNING: Undervoltage detected at Bus 2";
18
19 // Dependent print after tagged event
20 after "line_disconnect" is completed,
21    print "Line disconnect confirmed, starting recovery";
22
23 // Using now-relative timing
24 at now + 30 seconds, print "Midpoint checkpoint reached";
```

9 Best Practices

- Use consistent units (seconds, pu, Hz, probabilities in $[0, 1]$).
- Terminate every statement with `;`.
- Tag synchronization points for dependency chains.
- Validate interpolation laws and ensure interval bounds are consistent.
- Scope network/IIDM operators carefully (link vs. node), and prefer explicit tagging for auditability.
- Use `from` for deterministic ramps, otherwise the ramp starts from the current measurement.
- Prefer `open line` / `close line` for transmission line control over switch-based approaches for clarity.
- Use `print` statements to annotate key phases, decisions, or observations in scenarios—they aid debugging and make session replays self-documenting.

10 Function DSL Extension

This section documents the function extension available in the current interpreter. It is additive to the constructs documented above and introduces reusable action templates with invocation-time parameter binding.

10.1 Function Definition

Syntax:

```
func <NAME>(<PARAM_1>, <PARAM_2>, ...) {
   at +0 seconds, <EXECUTABLE>;
   at +5 seconds, <EXECUTABLE>;
   when <DYNAMIC_EXPRESSION>, <EXECUTABLE>;
   after "<TAG_ID>" is completed, <EXECUTABLE>;
}
```

Example:

```
func stagedLoadBoost(target, amount) {
   at +0 seconds, increase load target by amount% tagged "
   boost_start";
   at +5 seconds, decrease load target by amount% tagged "boost_end
   ";
}
```

Description:

- Function bodies contain `functionStatement` entries. Inside a function, `at` timestamps are relative to the function invocation time.
- A function body may contain ordinary actions, nested function calls, or `checkAndApply(...)` around a function call.
- `when` and `after` are also allowed inside a function body.
- Function definitions are simulation-scoped within the active interpreter session.

10.2 Function Invocation

Syntax:

```
<NAME>(<ARG_1>, <ARG_2>, ...);
<NAME>(param_1: <ARG_1>, param_2: <ARG_2>);
checkAndApply(<NAME>(...));
```

Examples:

```
stagedLoadBoost("_LOAD___2_EC", 5);

at 35 seconds, stagedLoadBoost("_LOAD___3_EC", 3);

at 42 seconds, checkAndApply(
   stagedLoadBoost(target: "_LOAD___4_EC", amount: 2)
);
```

Description:

- A direct call expands the function body into ordinary DSL events immediately.
- Direct calls may appear as bare statements or under `at`, `when`, and `after` triggers.
- Arguments may be positional or named. Named arguments bind by parameter name. Positional arguments must appear before named arguments.
- Parameters can be used in targets, numeric fields, booleans, time values, tags, and nested function-call arguments.

10.3 Relative Timing Rules

Examples:

```
func couplerPulse(target) {
   at +0 seconds, open switch target tagged "pulse_opened";
   after "pulse_opened" is completed, close switch target;
}

at 60 seconds, couplerPulse("VL_A_400_COUPLER_BREAKER");
```

Description:

- All explicit times inside a function body are relative to the invocation time.
- In the example above, the `open switch` action is scheduled at 60 seconds.
- Dependency-based statements such as `after "pulse_opened" is completed` remain relative to the events created by that invocation.

10.4 Verification with checkAndApply

Syntax:

```
checkAndApply(<FUNCTION_CALL>);
```

Example:

```
func plannedConsignation(breaker, coupler) {
   at +0 seconds, open switch breaker tagged "breaker_opened";
   at +20 seconds, couplerPulse(coupler);
}

at 150 seconds, checkAndApply(
   plannedConsignation(
      breaker: "VL_A_400_TRANSFO_1_BREAKER",
      coupler: "VL_A_400_COUPLER_BREAKER"
   )
);
```

Description:

- `checkAndApply(...)` performs a preflight expansion of the function call before persisting any generated events.
- If one generated action is invalid, unschedulable, or blocked by a higher-priority conflicting action, the entire function invocation is rejected.
- No partial application occurs: the expanded batch is persisted only if all checks pass.

10.5 Tags Inside Functions

Description:

- Tags declared inside a function invocation are local to that invocation.
- Repeated calls to the same function keep their internal tags isolated from one another.
- Tagging the function call itself creates an outer synchronization point that completes only after all child events of that invocation have completed.

10.6 Complete Function Example

```
set simulation duration to 120 seconds;
set time step to 1 seconds;

func stagedLoadBoost(target, amount) {
   at +0 seconds, increase load target by amount% tagged "boost_start";
   at +5 seconds, decrease load target by amount% tagged "boost_end";
}

func recoverySequence(target, amount) {
   at +0 seconds, stagedLoadBoost(target, amount);
   after "boost_end" is completed, increase load target by 1%;
}

at 20 seconds, stagedLoadBoost("_LOAD___2_EC", 5);
at 35 seconds, checkAndApply(stagedLoadBoost(target: "_LOAD___3_EC", amount: 3));
after "line_disconnect" is completed, recoverySequence("_LOAD___4_EC", 2) tagged "recovery_done";
```